

Análisis de Series de Tiempo II

# Baselines Modernos y Feature Engineering Temporal

MSc. Fernando Silva



# Agenda

- Repaso
- Tipos de Baselines Modernos
- Introducción a Feature Engineering Temporal

Repaso

# Series de Tiempo

## Definición

Una serie de tiempo es una secuencia de observaciones o mediciones registradas en diferentes instantes de tiempo. Formalmente:

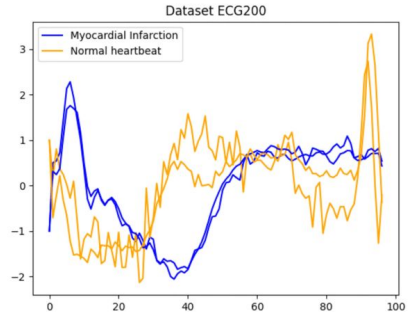
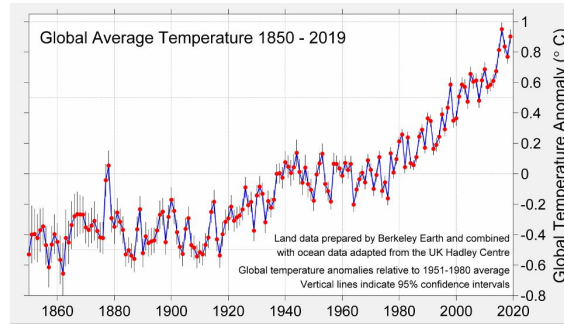
$X$  es la variable observada

$X(t_i)$

$t_i$  es el instante temporal (segundos, días, meses, etc)

Ejemplos:

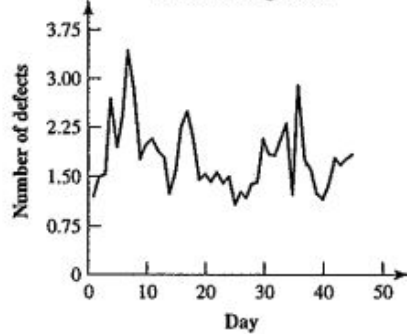
- Precio diario del dólar
- Temperatura cada hora
- Señal de audio
- Tráfico de red
- ECG (electrocardiograma)



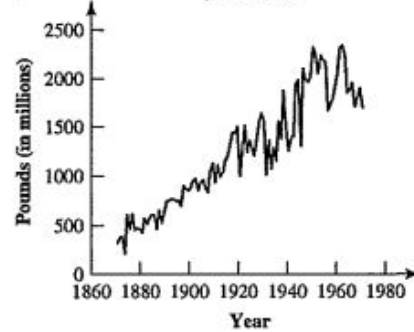
# Series de Tiempo

## Ejemplos

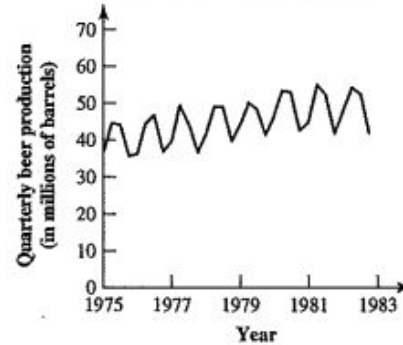
(a) Daily average number of truck manufacturing defects



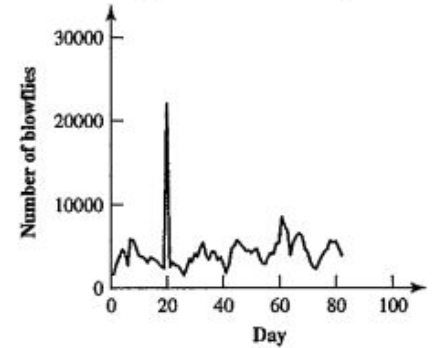
(b) Yearly U.S. tobacco production



(c) Quarterly U.S. beer production

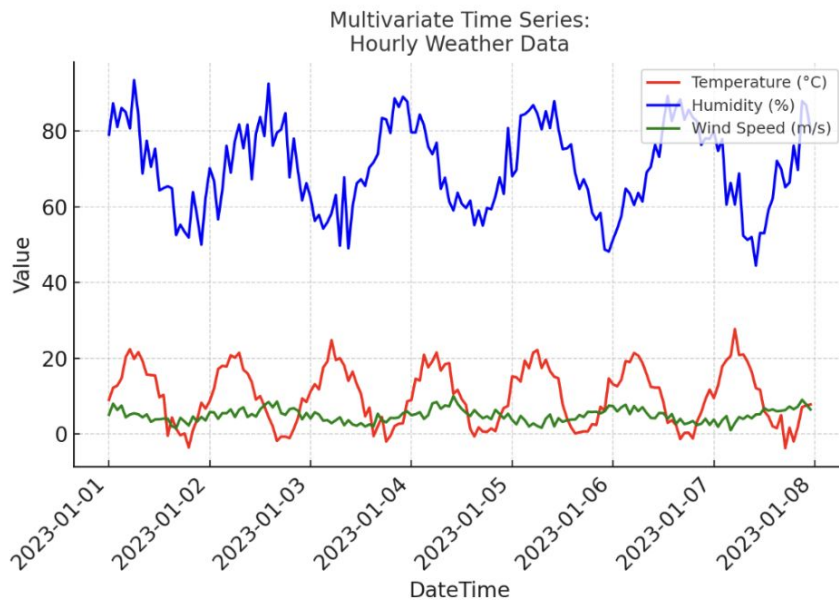
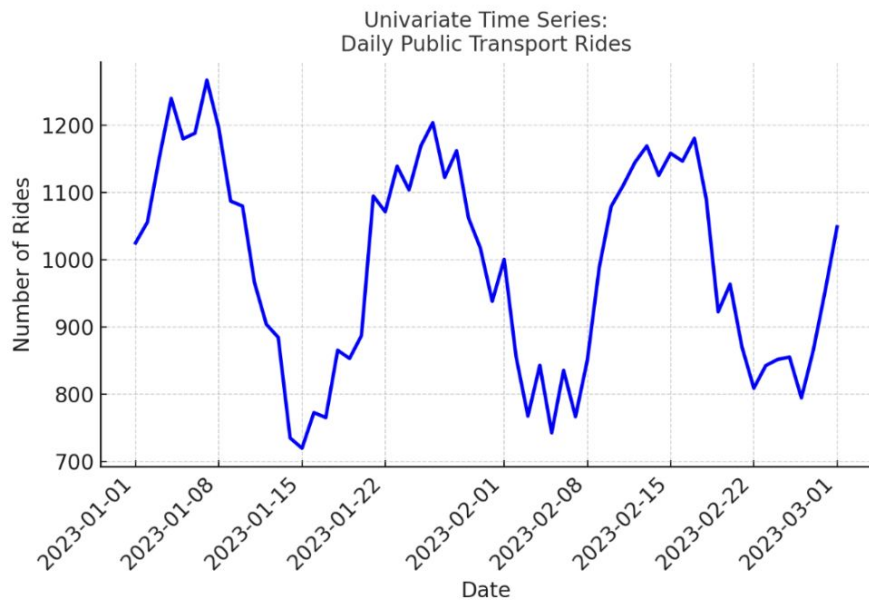


(d) Contaminated blowfly data



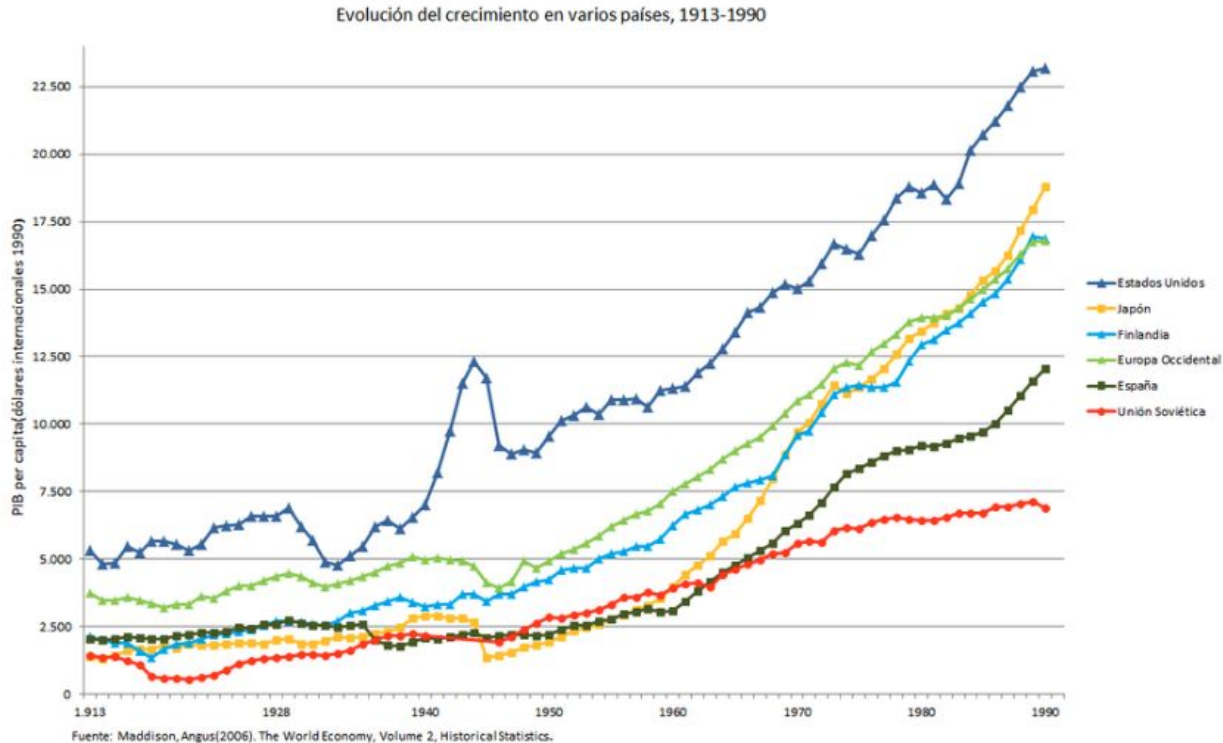
# Representación Temporal

## Series Univariadas vs Multivariadas



# Representación Temporal

## Panel data (múltiples series)



Referencia: Wikipedia, PIB per cápita 1920-1990

# Representación Temporal

## Regular vs Irregular

$\Delta t$  es el intervalo de tiempo entre dos observaciones consecutivas de la serie

### Regular

- Una serie regular la muestreamos a intervalos fijos: cada hora, cada día, cada minuto.
- Eso nos da un índice entero limpio y acceso directo a toda la maquinaria clásica: FFT para frecuencias, ARIMA, STL para descomposición estacional, autocorrelación, etc.

### Irregular

- Es el mismo fenómeno de fondo, pero observado a tiempos arbitrarios ( $\Delta t$  no constante).
- El problema: la mayoría de los modelos asumen  $\Delta t$  fijo.
- Dos salidas: regularizar a una grilla fija (imputar/resamplear) y usar modelos clásicos, o usar modelos que aceptan tiempo continuo (Neural ODE, o  $\Delta t$  como feature).

Ejemplos:

- Temperatura horaria
- Precio de cierre diario

Ejemplos:

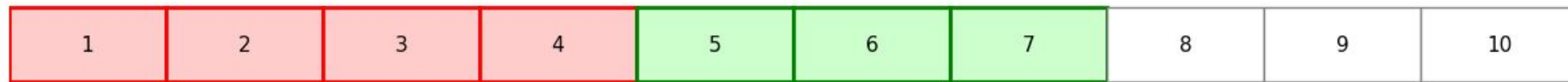
- Historial clínico paciente
- Lecturas de sensor IoT



# Conversión a Problema Supervisado

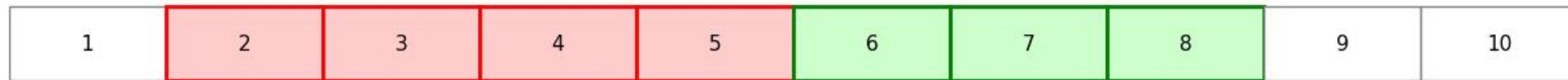
## Sliding Window

### Sliding Window



Input Window

Forecast Horizon



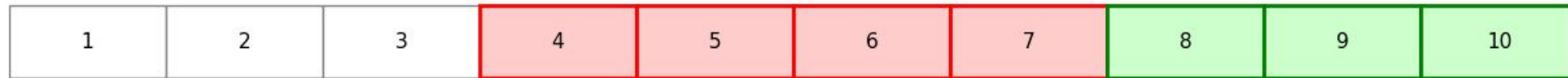
Input Window

Forecast Horizon



Input Window

Forecast Horizon



Input Window

Forecast Horizon

# Conversión a Problema Supervisado

## Rolling windows

Supongamos que tenemos la siguiente serie:

**10, 20, 30, 40, 50** y definimos ventana de tamaño **3**

### Sliding Window

Generamos muestras supervisadas (X, y)

$X = [10, 20, 30], y = 40$

$X = [20, 30, 40], y = 50$

**sliding lo usamos para armar pares  
(X, y)**

### Rolling Window

Calculamos una estadística que “rueda” sobre la serie. Misma ventana, pero la salida es un solo número agregado por posición

$\text{media}(10, 20, 30) = 20$

**rolling lo usamos para resumir  
(feature engineering)**

# Conversión a Problema Supervisado

## Expanding windows

- La ventana de input **crece**, el horizonte es fijo.
- Esto reaparece en validación (walk-forward).



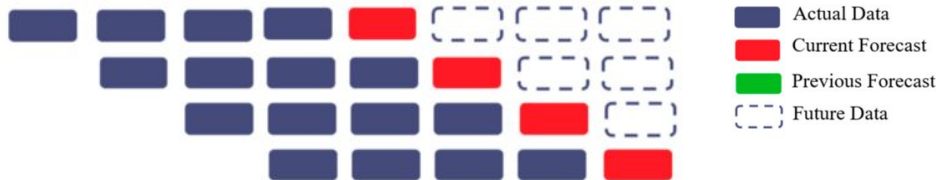
# Tipos de Forecasting

## One-Step Ahead vs Multi-Step Ahead

La diferencia fundamental es el horizonte H: cuántos pasos futuros quieres predecir de una sola vez.

### One-Step Ahead (H=1)

- El modelo solo predice el siguiente valor. No hay forma de que el error se propague porque no hay pasos encadenados.
- Ideal cuando se necesita respuesta inmediata y precisa, como detección de anomalías en tiempo real o control de procesos industriales.



### Multi-Step Ahead (H>1)

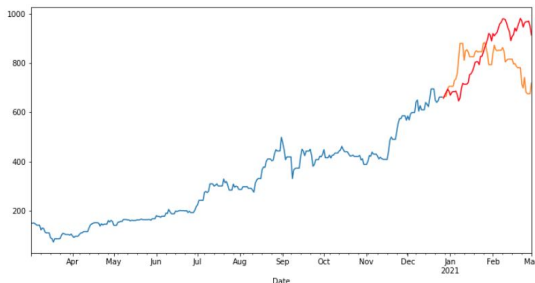
- Predice varios pasos futuros, y aquí aparece el problema central: la acumulación de error.
- Cuanto más lejos en el horizonte, mayor la incertidumbre.



# Introducción al Forecasting Moderno

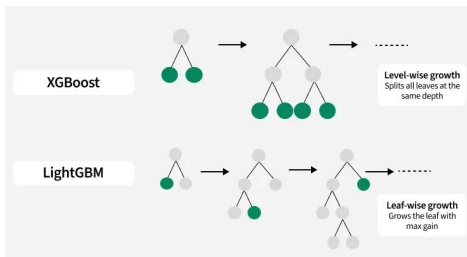
## Clásico Estadístico (ARIMA, ETS, Theta, Prophet)

Modelo generativo  
explícito, una serie a la vez,  
funciona con pocos datos.



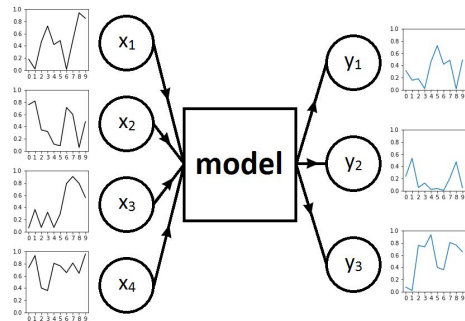
## ML tabular (LightGBM/XGBoost sobre features)

La serie se vuelve tabla, es  
global, depende de feature  
engineering manual, y suele  
ser el elegido para  
producción.



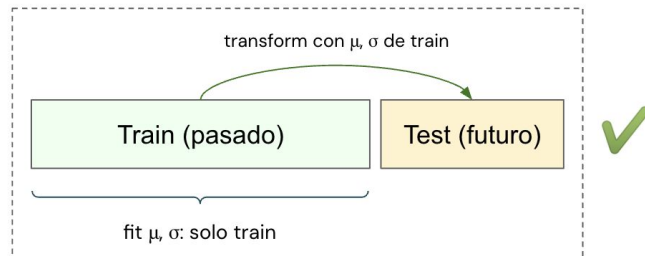
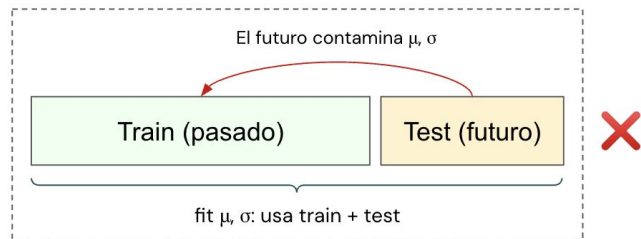
## Deep Learning (RNN/LSTM/TCN/ Transformers)

Aprende la representación,  
global, escala con datos,  
caja negra.



# Leakage Temporal

## Leakage por Escalado

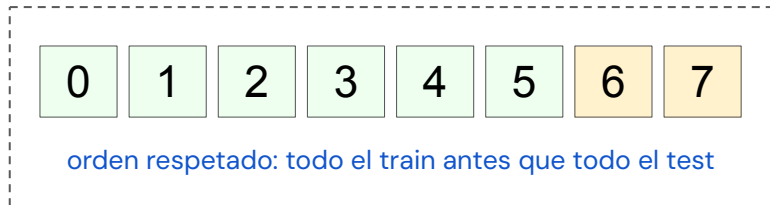
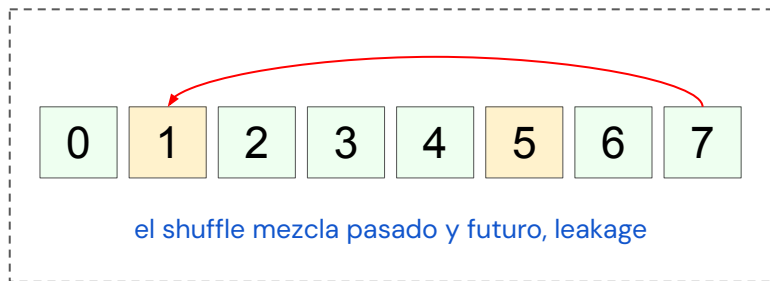


## Leakage por Ventana

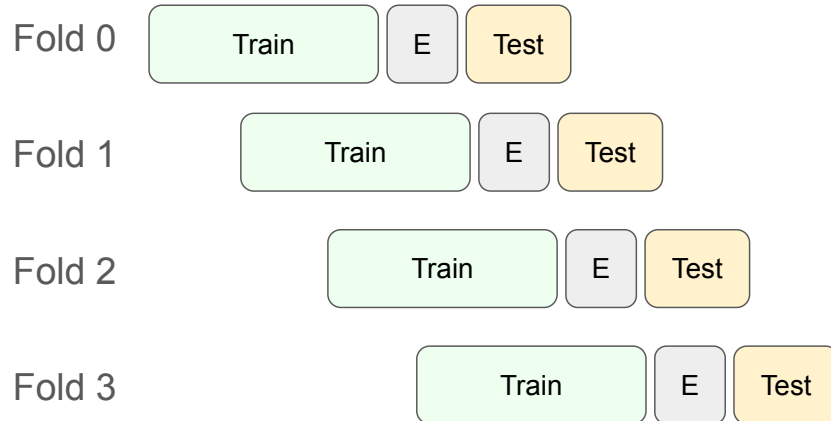


# Validación Temporal

## Holdout Temporal



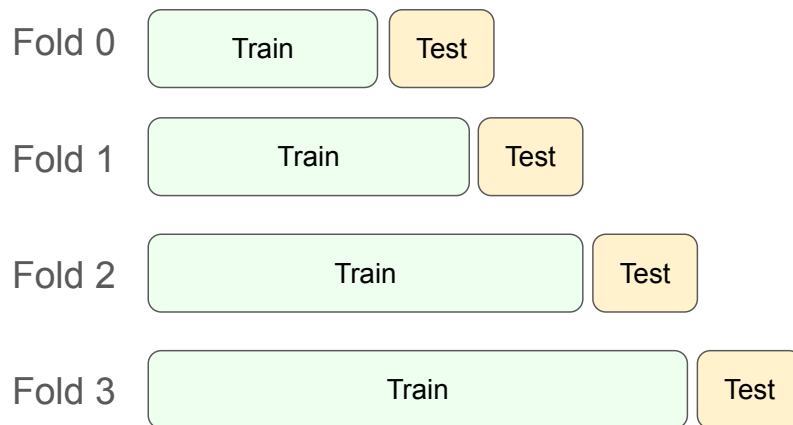
## Walk-forward



E (Embargo) es la zona de descarte entre train y test para que ninguna ventana de input cruce el corte

# Validación Temporal

## Expanding Window





# Introducción

# ¿Por qué los baselines siguen siendo críticos?

## ¿Qué es un baseline?

- Un baseline es un modelo trivial/barato que define el piso de desempeño que cualquier modelo serio debe superar.
- En producción, muchos problemas se resuelven con baselines robustos
- Benchmarking antes de deep learning / boosting

*Si no se supera un baseline robusto, el modelo complejo no aporta valor.*

**Naive forecast**



Referencia: <https://towardsdatascience.com/baseline-models-in-time-series-c76d44a826b3/>

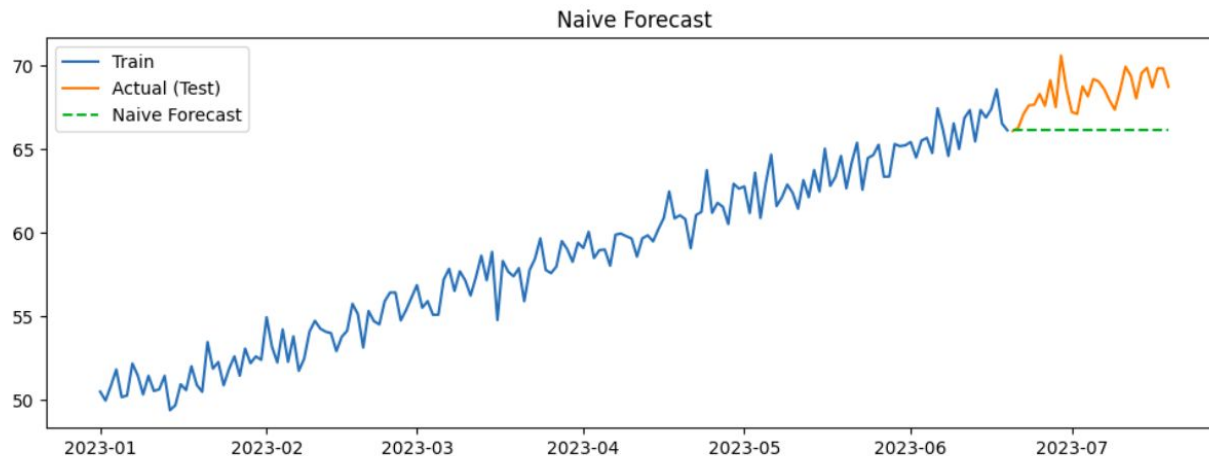
# Tipos de Baselines Modernos

# Tipos de Baselines Modernos

## Naive Baseline

Consiste en usar el último valor observado:

$$\hat{y}_{t+1} = y_t$$



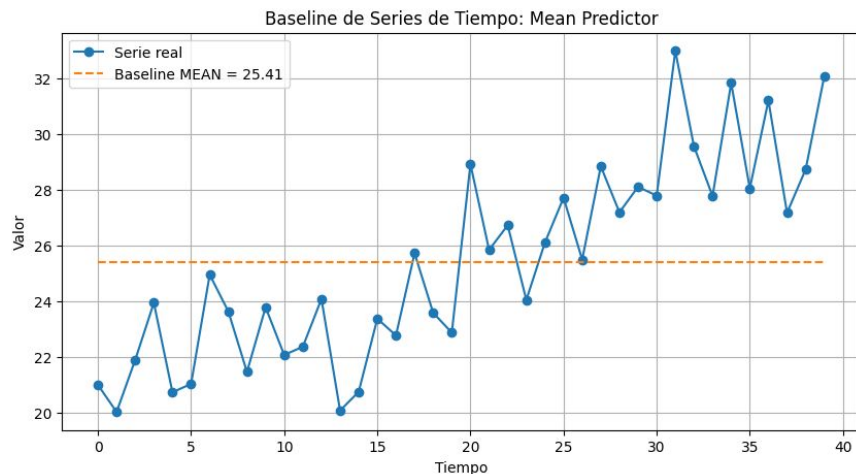
Referencia: <https://medium.com/coding-nexus/time-series-chapter-6-introduction-to-forecasting-na%C3%AFve-mean-drift-models-lae4269d5481>

# Tipos de Baselines Modernos

## Mean

Consiste en predecir siempre el promedio histórico observado:

Si tiene la serie temporal:  $y_1, y_2, y_3, \dots, y_t$  el Mean Baseline predice:  $\hat{y}_{t+1} = \frac{1}{t} \sum_{i=1}^t y_i$



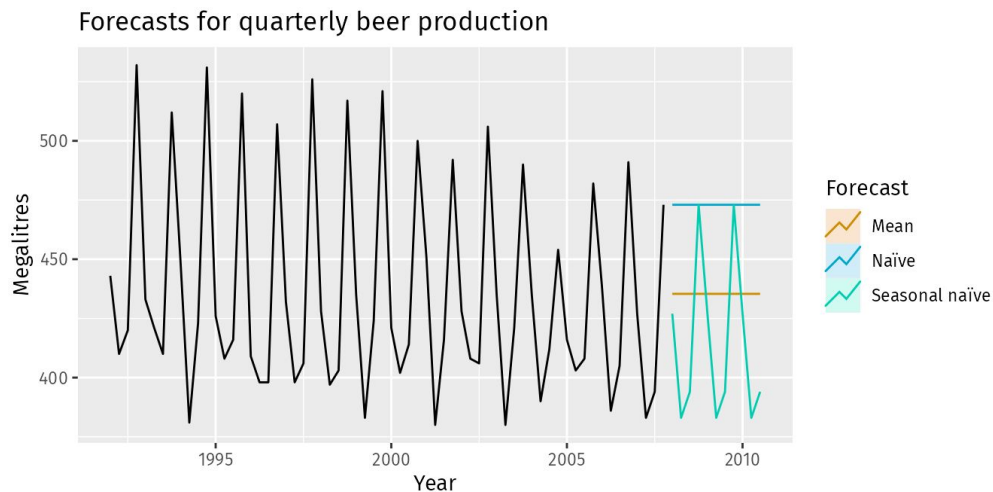
Útil cuando hay ruido sin estructura.

# Tipos de Baselines Modernos

## Seasonal Naive Baseline

Usa el valor del mismo punto de la estación anterior (mismo día de la semana pasada, mismo mes del año pasado)

$$\hat{y}_{t+h} = y_{t+h-m}$$



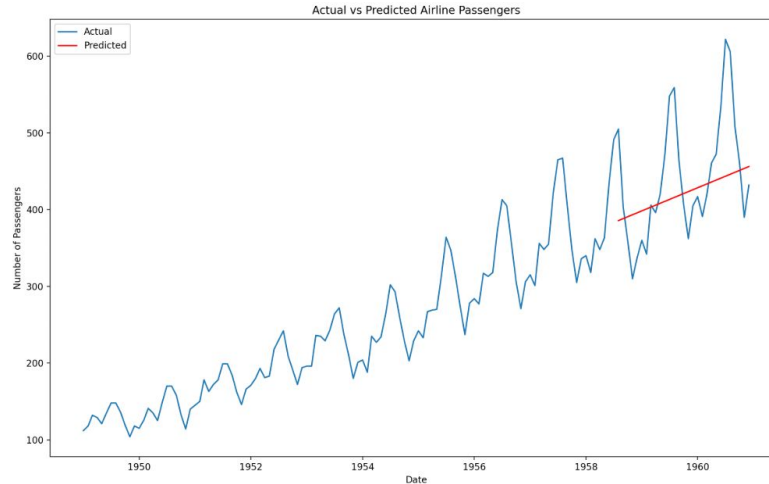
Útil cuando hay  
estacionalidad fuerte.

# Tipos de Baselines Modernos

## Linear Trend Baseline

Modelo lineal simple:

$$y = \beta_0 + \beta_1 t$$



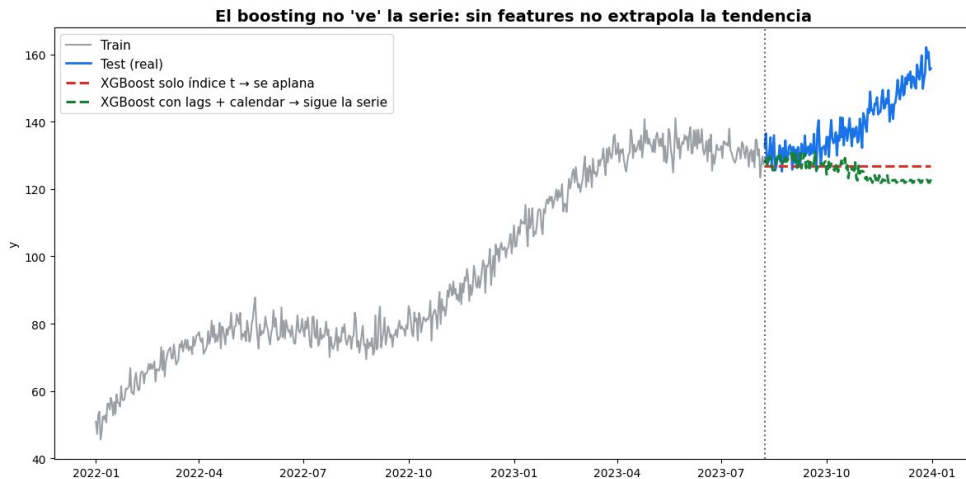
Referencia: <https://www.geeksforgeeks.org/machine-learning/step-by-step-guide-to-modeling-time-series-data-using-linear-regression/>

Útil cuando la serie  
tiene tendencia

# Tipos de Baselines Modernos

## Gradient Boosting Baseline

- Hoy el baseline "serio" en datos tabulares suele ser boosting.
- Pero boosting no ve la serie: no tiene noción de orden ni de tiempo. Trata cada fila como un ejemplo i.i.d.
- Hay que convertir la serie en tabla: cada fila = un instante; columnas = lags + rolling + calendar features; target =  $y_t$ .
- Sin feature engineering, XGBoost sobre el índice temporal no aporta nada.



El poder no está en el modelo, está en los features.



# Tipos de Baselines Modernos

## Métricas

### MAE (Mean Absolute Error)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Error promedio en unidades originales.

Interpretable, robusto a outliers.

No castiga extra los errores grandes.

### RMSE (Root Mean Squared Error)

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Penaliza cuadráticamente

Útil cuando los errores grandes cuestan desproporcionado.

Sensible a outliers.

### MAPE

(Mean Absolute Percentage Error)

$$MAPE = \frac{\sum \frac{|A-F|}{A} \times 100}{N}$$

Error en %, escala-libre

Fácil de comunicar

Inútil en demanda intermitente o series que pasan por cero

### MASE

(Mean Absolute Scaled Error)

$$MASE = \frac{MAE}{\frac{1}{n-1} \sum_{i=2}^n |y_i - y_{i-1}|}$$

Escala libre y relativa al naive.

La única que te dice si el modelo aporta valor real.

**A** = Actual (valor real/observado)

**F** = Forecast (valor pronosticado)

# Baselines Modernos

[LINK](#)



# Ejercicio Baselines Modernos

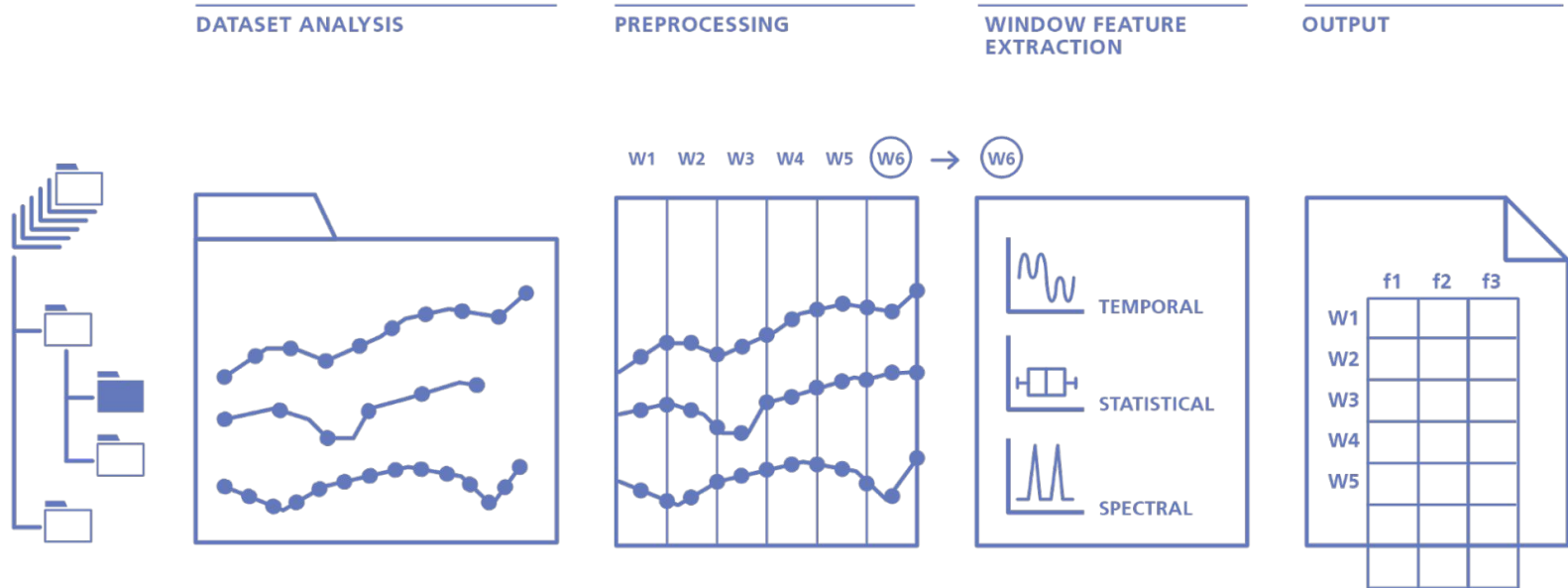
[LINK](#)



# Introducción a Feature Engineering Temporal

# Introducción a Feature Engineering Temporal

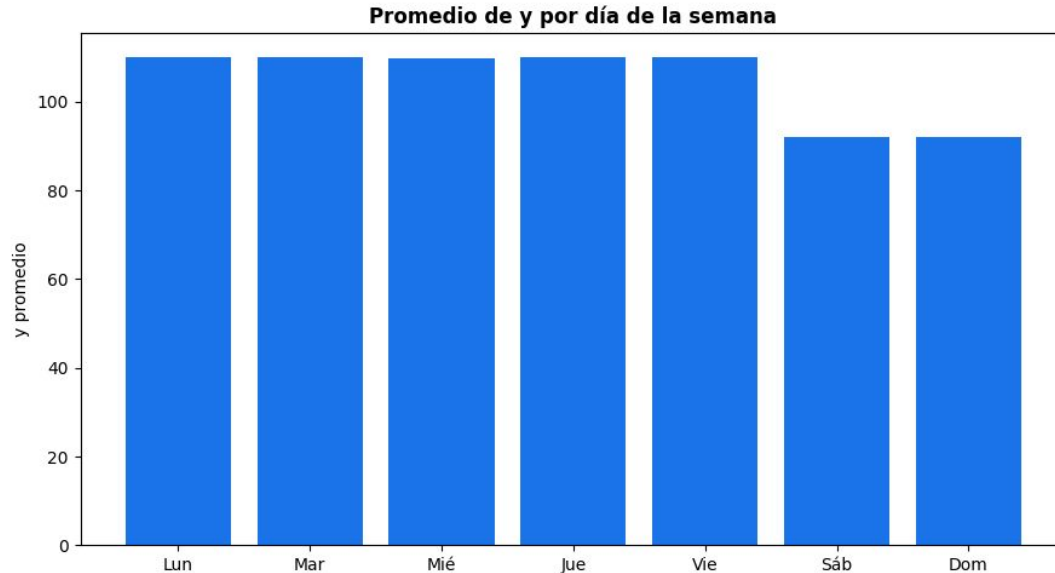
¿Cómo extraer señal temporal sin mirar el futuro?



# Time-based Features (Calendar Features)

## Día de la semana

- Captura la estacionalidad semanal.
- Útil en demanda, tráfico, consumo: el comportamiento de un lunes no es igual al de un domingo.

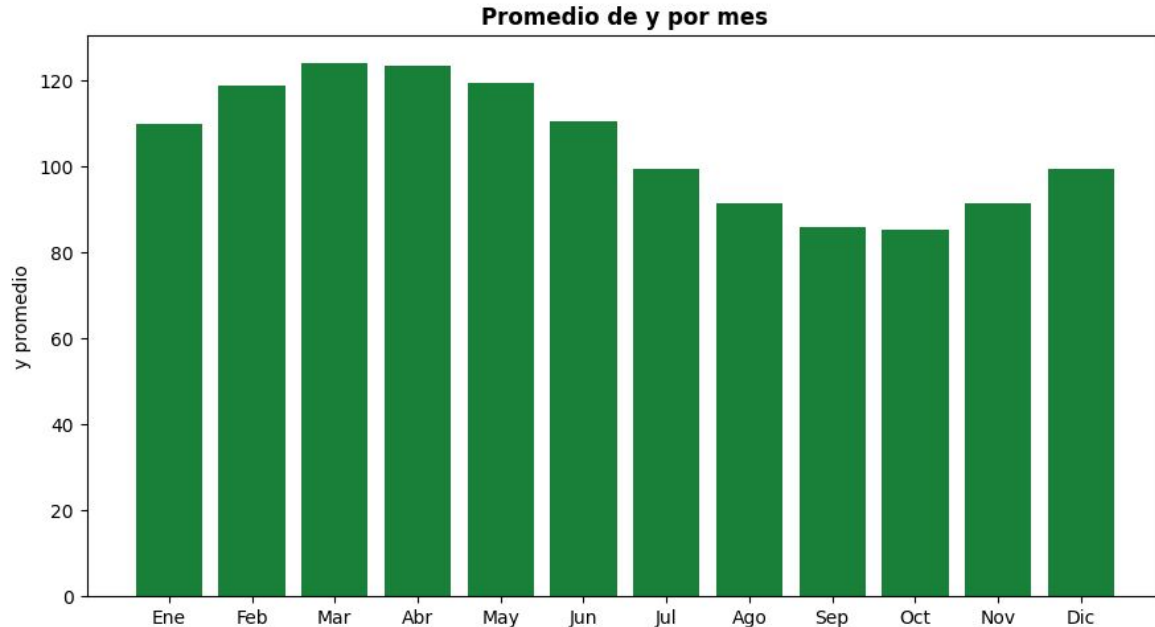


Se ven 5 barras altas (laborables) y 2 bajas (finde): el patrón salta a la vista.

# Time-based Features (Calendar Features)

## Mes (Seasonality)

- Captura la estacionalidad anual.
- Clásico en retail, energía, turismo.

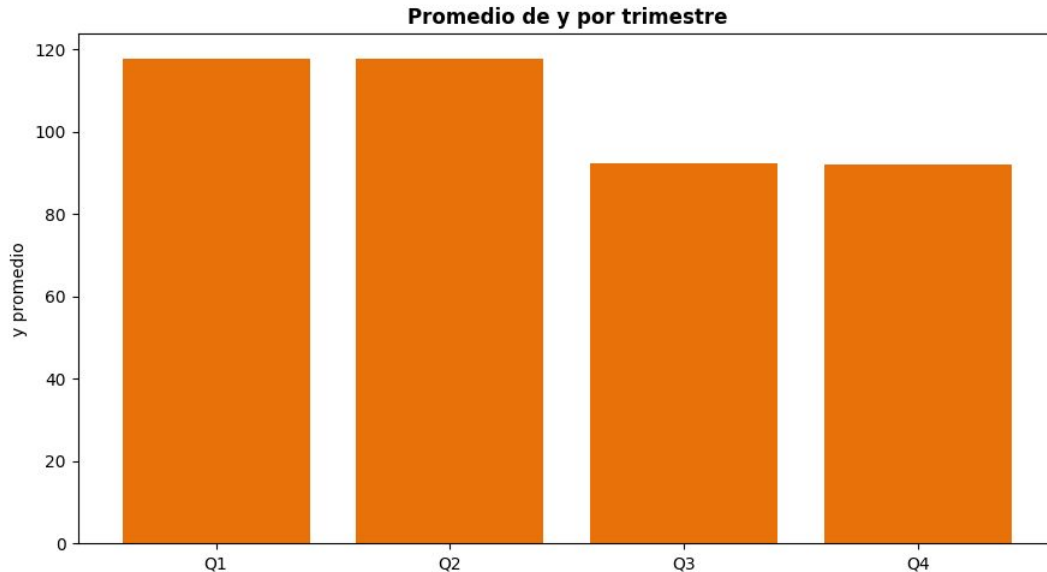


La curva sigue la senoide anual: subida hacia el centro del año y caída en los extremos.

# Time-based Features (Calendar Features)

## Quarter

- Misma estacionalidad anual pero a resolución más gruesa.
- Útil cuando los datos son ruidosos a nivel mensual o cuando el negocio razona por trimestres (reportes financieros).



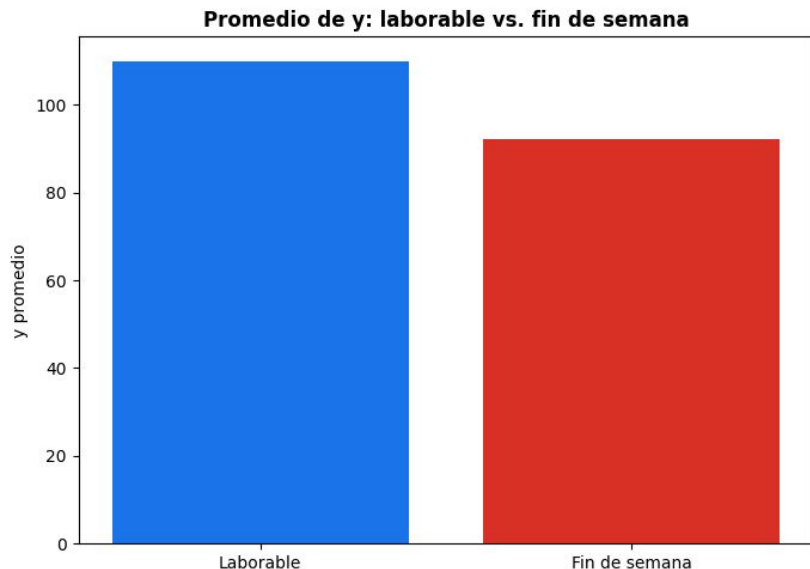
Quarter es month "agrupado", menos granularidad, menos features, menos riesgo de overfitting, pero también menos detalle.



# Time-based Features (Calendar Features)

## Fin de semana

- Feature binaria derivada del día de la semana.
- A veces lo único que importa no es qué día es, sino si es laborable o no.
- Una sola columna en vez de siete.

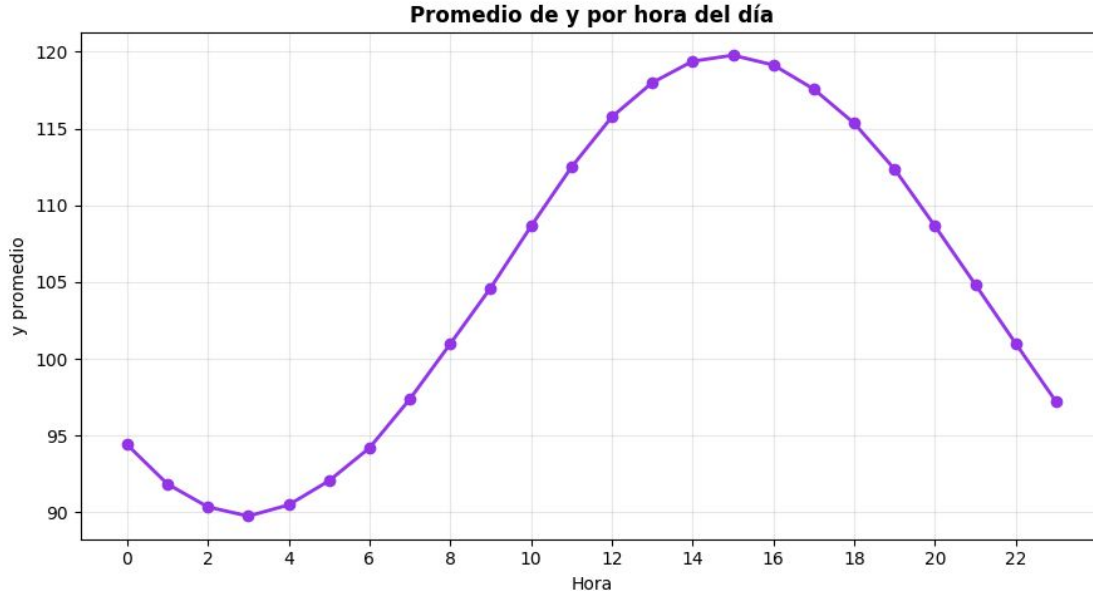


Contraste con “Día de la semana”: misma información comprimida en 1 feature. Trade-off de granularidad otra vez.

# Time-based Features (Calendar Features)

## Hora del día

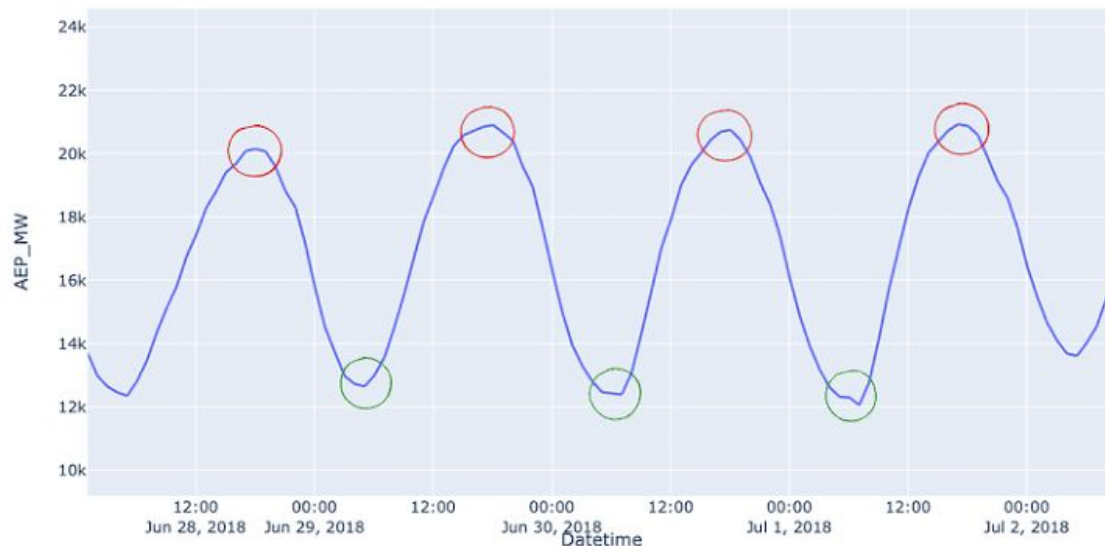
- Captura la estacionalidad intradía.
- Imprescindible en energía, tráfico web, transporte, donde el ciclo de 24 h domina.



La hora es continua y cíclica, y la curva deja ver el pico de la tarde.

# Encoding Cíclico

- Problema: hora (0–23), mes (1–12), día de semana (0–6) son cíclicas. La distancia entre las 23h y las 0h es 1 hora, no 23. Si las pasamos como entero, el modelo cree que están lejos..



Importa cuando los modelos que asumen distancia/continuidad (lineales, redes neuronales).

Para árboles/boosting el beneficio es marginal (los splits manejan bien los enteros)

# Holidays / Eventos

- Las features de calendario regulares capturan estacionalidad periódica. Pero **los picos más grandes en demanda real son eventos no periódicos:** feriados, promos, Black Friday, clima, eventos deportivos.
- En retail/energía/demanda, **suelen ser las features de mayor valor predictivo, por encima de día de semana.**
- Tipos:
  - Binaria: es feriado (0/1)
  - Distancia al evento: días hasta/desde el próximo feriado (captura el build-up pre-navidad y la resaca después)
  - Categórica: qué feriado es (no todos pesan igual)

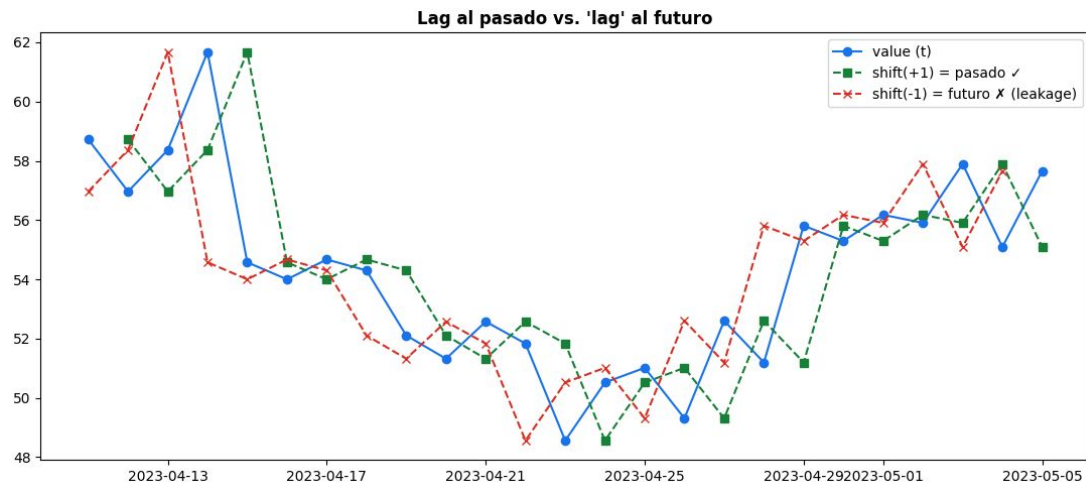
# Feature Engineering, Parte 1

[LINK](#)



# Lag Features

- Un lag es el valor de la serie  $k$  pasos atrás, convertido en columna.
- Es la feature más básica y más potente: le da a un modelo tabular la memoria que no tiene.



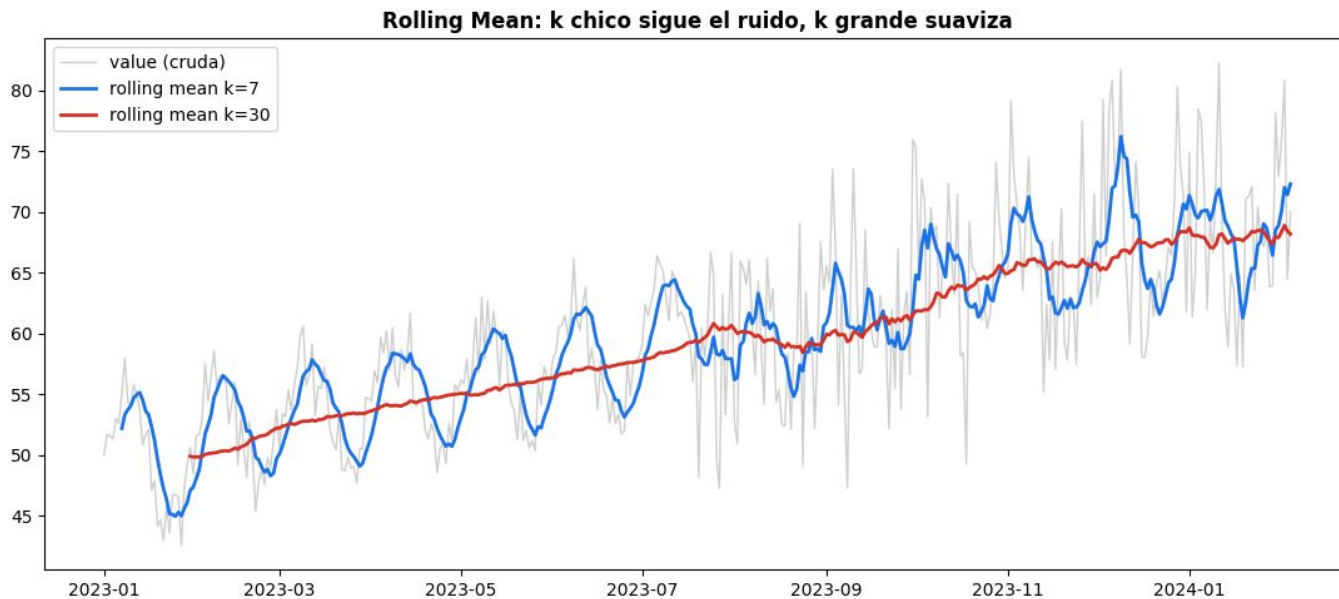
| $t$ | value | lag1<br>value(t-1) | lag2<br>value(t-2) |
|-----|-------|--------------------|--------------------|
| 1   | 5     | NaN                | NaN                |
| 2   | 10    | 5                  | NaN                |
| 3   | 15    | 10                 | 5                  |

La línea verde está desfasada hacia la derecha (usa lo de ayer), la roja hacia la izquierda (usa lo de mañana). Misma operación, signo opuesto, una sola es legal.

# Rolling Window Features

## Rolling Mean

- Promedio de las últimas k observaciones.
- Captura el nivel suavizado/tendencia local, filtrando el ruido de alta frecuencia.

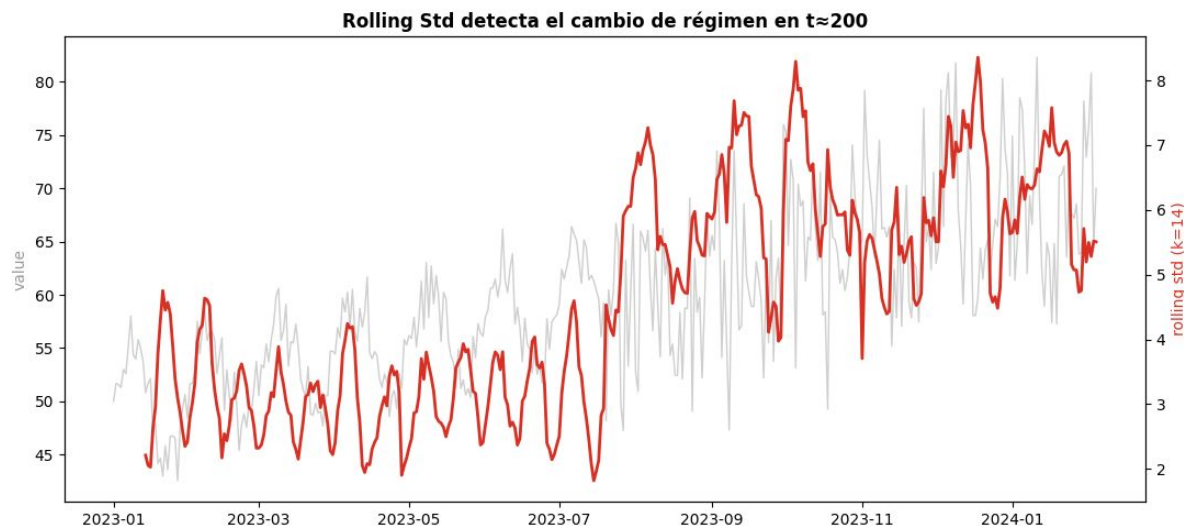


$$rolling\_mean_t = \frac{1}{k} \sum_{i=1}^k y_{t-i}$$

# Rolling Window Features

## Rolling Std

- Desvío estándar móvil.
- Captura la volatilidad local, cuánto se está moviendo la serie ahora.
- Útil para detectar regímenes de incertidumbre (mercados, fallas, picos de demanda).



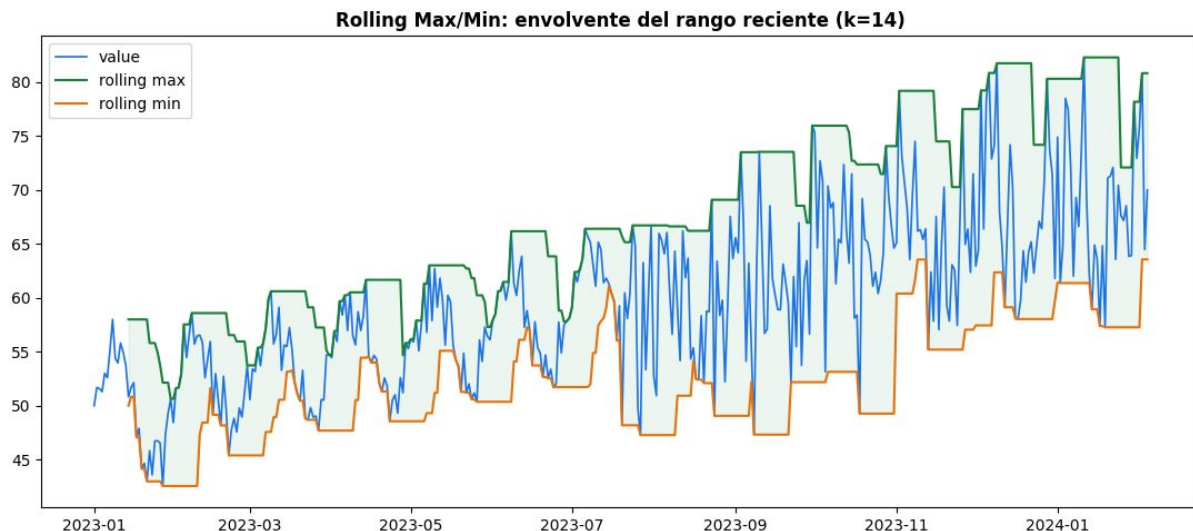
El std se dispara justo donde subimos la volatilidad: deja ver que la feature "sabe" cuándo la serie se vuelve inestable.



# Rolling Window Features

## Rolling Max / Min

- Máximo y mínimo móviles.
- Definen una banda/envolvente del rango reciente.
- Capturan el régimen (¿estamos cerca de un techo o un piso histórico reciente?) y picos.



Sobre la elección de k:

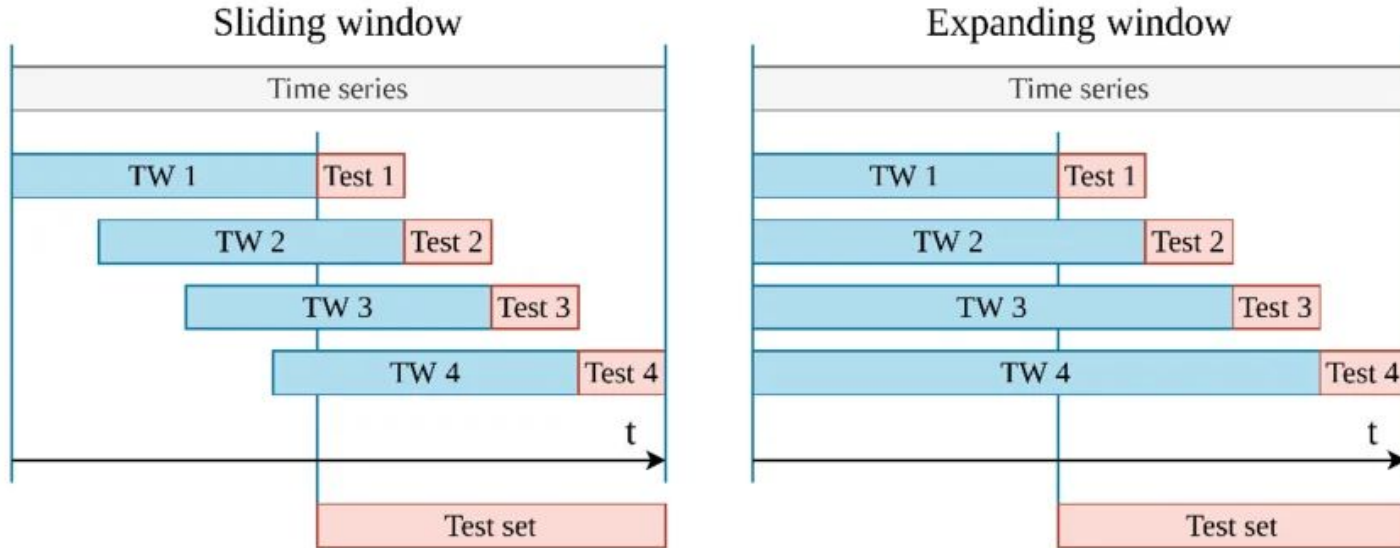
- k chico = feature reactiva pero ruidosa
- k grande = estable pero con lag.

No hay valor mágico: se prueba (7, 14, 30...) y se valida.

# Expanding Window Features

- Consiste en la acumulación histórica, por ejemplo el promedio de todo el historial previo:

$$expanding\_mean_t = \frac{1}{t-1} \sum_{i=1}^{t-1} y_i$$



Expanding acumula todo el historial

# Feature Engineering, Parte 2

[LINK](#)



# Feature Engineering, Ejercicio

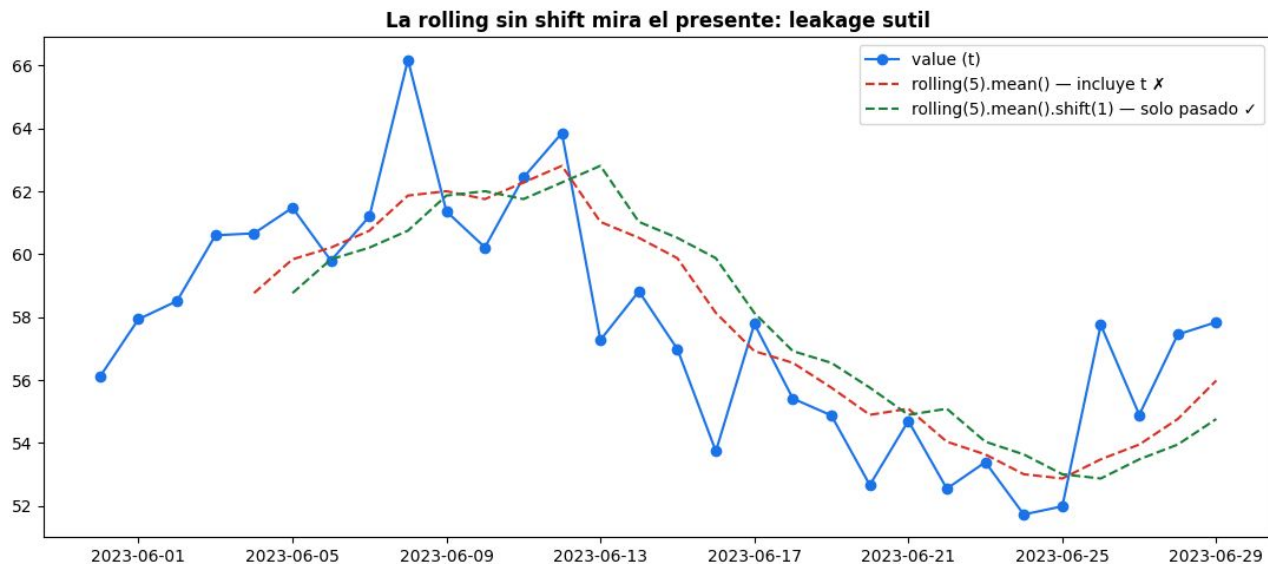
[LINK](#)



Dataset Melbourne

# Leakage en Feature Engineering Temporal

- El feature engineering temporal es la fuente de leakage más sigilosa, porque las funciones de pandas por defecto incluyen el instante actual.



Regla: una feature solo puede usar datos anteriores al timestamp del target.

En este setup tabular la fila  $t$  predice  $y_t$ , así que las features quedan en  $\leq t-1$ .

Si en producción no la tendrías a tiempo, es leakage.

# Trade-offs

## Tamaño de ventana

- corta = reactiva pero ruidosa
- larga = estable pero con lag.
  - No existe el k universal.

## Costo computacional

- Lags y rollings son baratísimos.
- DL necesita GPU y tuning.

## Cantidad de features

- Más lags/rollings = más señal, pero también más riesgo de overfitting y más NaN al inicio.
  - Más no siempre es mejor.

## Interpretabilidad

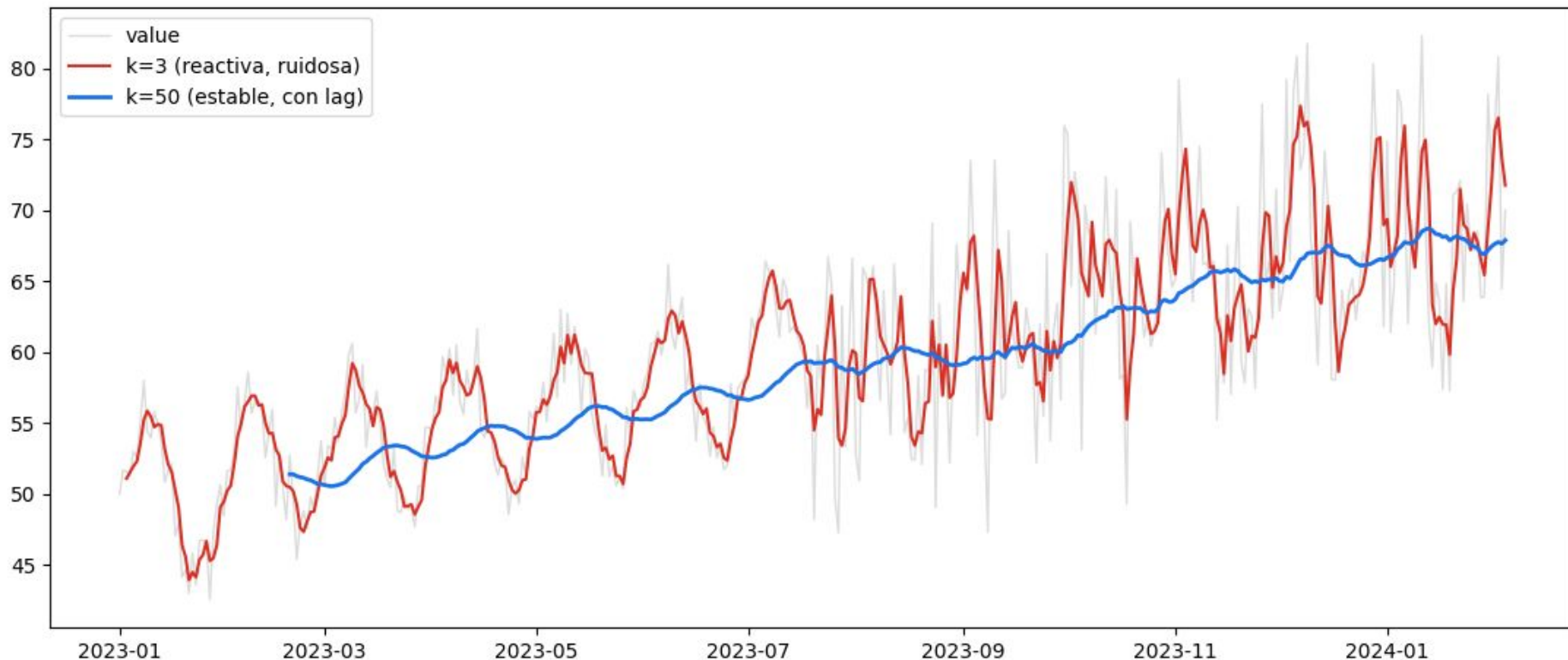
- FE + boosting es transparente.
- Deep Learning es caja negra.

- Feature engineering + boosting le gana a deep learning la mayoría de las veces, más rápido, más barato y más interpretable.
- El DL recién paga cuando hay muchísimos datos y dependencias muy complejas.

# Trade-offs

## Tamaño de ventana

**Trade-off de la ventana: reactividad vs. estabilidad**



# Resumen

## El baseline es el piso, no un trámite

Si tu modelo no le gana a un naive / seasonal naive robusto, no aporta valor.

## Tres familias de features temporales

Calendario (estacionalidad), lags (memoria) y ventanas, rolling / expanding / EWM, (nivel, volatilidad, régimen).

## Comparar siempre contra el naive

MAE / RMSE / MAPE describen el error; MASE te dice si el modelo realmente aporta sobre el baseline.

## El poder no está en el modelo, está en los features

Boosting no ve el tiempo: hay que dárselo. La serie se vuelve tabla con lags + rolling + calendar features.

## Toda feature mira solo el pasado

`rolling().shift(1)`, nunca `rolling()` a secas: el default de pandas filtra el presente.

## No hay k universal ni "más es mejor"

Ventana corta = reactiva pero ruidosa, larga = estable pero con lag.  
Más features = más señal, pero más overfitting y más NaN.

**El DL recién funciona muchísimos datos y dependencias largas y no lineales**  
**Esto lo veremos la siguiente clase.**